Use monoids to compute a "bag" from an `IndexedSeq`.

```
def bag[A](as: IndexedSeq[A]): Map[A, Int]
```

---

### 10.6.2 Using composed monoids to fuse traversals

The fact that multiple monoids can be composed into one means that we can perform multiple calculations simultaneously when folding a data structure. For example, we can take the length and sum of a list at the same time in order to calculate the mean:

```
scala> val m = productMonoid(intAddition, intAddition)
m: Monoid[(Int, Int)] = $anon$1@8ff557a

scala> val p = listFoldable.foldMap(List(1,2,3,4))(a => (1, a))(m)
p: (Int, Int) = (4, 10)

scala> val mean = p._1 / p._2.toDouble
mean: Double = 2.5
```

It can be tedious to assemble monoids by hand using `productMonoid` and `foldMap`. Part of the problem is that we're building up the `Monoid` separately from the mapping function of `foldMap`, and we must manually keep these "aligned" as we did here. But we can create a combinator library that makes it much more convenient to assemble these composed monoids and define complex computations that may be parallelized and run in a single pass. Such a library is beyond the scope of this chapter, but see the chapter notes for a brief discussion and links to further material.

## 10.7 Summary

Our goal in part 3 is to get you accustomed to working with more abstract structures, and to develop the ability to recognize them. In this chapter, we introduced one of the simplest purely algebraic abstractions, the monoid. When you start looking for it, you'll find ample opportunity to exploit the monoidal structure of your own libraries. The associative property enables folding any `Foldable` data type and gives the flexibility of doing so in parallel. Monoids are also compositional, and you can use them to assemble folds in a declarative and reusable way.

`Monoid` has been our first purely abstract algebra, defined only in terms of its abstract operations and the laws that govern them. We saw how we can still write useful functions that know nothing about their arguments except that their type forms a monoid. This more abstract mode of thinking is something we'll develop further in the rest of part 3. We'll consider other purely algebraic interfaces and show how they encapsulate common patterns that we've repeated throughout this book.